

Pandas: A 10-Minute Guide (80% Coverage)

Introduction

Pandas is Python's go-to library for working with tabular data. Two core structures: **Series** (1D) and **DataFrame** (2D). Think of it as Excel for Python.

1. Creating Data Structures

Series (1D Array)

```
import pandas as pd

# From a list
s = pd.Series([10, 20, 30, 40])
print(s)
# Output: Index 0-3, Values 10-40

# With custom index
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s['a']) # Output: 10
```

DataFrame (2D Table)

```
# From a dictionary
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'salary': [50000, 60000, 70000]
})

# From a CSV file
df = pd.read_csv('data.csv')

# From a list of lists
df = pd.DataFrame([[ 'Alice', 25], [ 'Bob', 30]], columns=['name', 'age'])
```

2. Inspecting Data

```
df.head()           # First 5 rows
df.tail(3)          # Last 3 rows
df.shape            # (rows, columns)
df.info()           # Data types, missing values, memory
df.describe()       # Statistics (mean, std, min, max)
df.dtypes           # Column data types
df.columns          # Column names
df.index            # Row labels
```

3. Selecting & Accessing Data

Column Access

```
df['name']          # Single column (Series)
df[['name', 'age']] # Multiple columns (DataFrame)
df.name             # Dot notation (only for valid names)
```

Row Access

```
df.loc[0]           # By label (index)
df.iloc[0]          # By position (0-based)
df.loc[0, 'name']    # Specific cell
```

Slicing

```
df[0:3]             # First 3 rows
df.loc[0:2, 'name':'age'] # Rows 0-2, columns 'name' to 'age'
```

4. Filtering & Querying

```
# Boolean indexing
df[df['age'] > 25]          # Rows where age > 25

# Multiple conditions
df[(df['age'] > 25) & (df['salary'] > 55000)]

# Using query() (more readable)
df.query('age > 25 and salary > 55000')

# Using isin()
df[df['name'].isin(['Alice', 'Bob'])]
```

5. Adding & Modifying Columns

```
# Add a new column
df['bonus'] = 5000

# Column from calculation
df['annual'] = df['salary'] * 12

# Using apply() for custom function
df['name_upper'] = df['name'].apply(str.upper)

# Drop column
df = df.drop('bonus', axis=1)

# Rename column
df = df.rename(columns={'salary': 'annual_salary'})
```

6. Handling Missing Data

```
# Check for NaN
df.isnull()          # Boolean DataFrame
df.isnull().sum()     # Count of missing values per column

# Remove rows with missing values
df = df.dropna()

# Fill missing values
df['age'] = df['age'].fillna(0)          # Fill with 0
df['age'] = df['age'].fillna(df['age'].mean()) # Fill with mean

# Forward fill (use previous value)
df['age'] = df['age'].fillna(method='ffill')
```

7. Sorting & Ranking

```
# Sort by one column
df.sort_values('age')

# Sort by multiple columns
df.sort_values(['age', 'name'])

# Descending order
df.sort_values('age', ascending=False)

# Sort index
df.sort_index()

# Rank (1, 2, 3, ...)
df['rank'] = df['salary'].rank()
```

8. Grouping & Aggregation

```
# Group and aggregate
df.groupby('department')['salary'].mean()

# Multiple aggregations
df.groupby('department').agg({
    'salary': ['mean', 'sum', 'count'],
    'age': 'mean'
})

# Custom aggregation
df.groupby('department')['salary'].agg(['min', 'max', 'std'])

# Apply function to groups
df.groupby('department').apply(lambda x: x['salary'].max())
```

9. Merging & Joining

```
# Merge two DataFrames (SQL-like JOIN)
df1 = pd.DataFrame({'id': [1, 2], 'name': ['Alice', 'Bob']})
```

```
df2 = pd.DataFrame({'id': [1, 2], 'salary': [50000, 60000]})

merged = pd.merge(df1, df2, on='id') # Inner join (default)
merged = pd.merge(df1, df2, on='id', how='left') # Left join
merged = pd.merge(df1, df2, on='id', how='outer') # Full outer join

# Concatenate (stack vertically)
result = pd.concat([df1, df2], axis=0)

# Concatenate horizontally
result = pd.concat([df1, df2], axis=1)
```

10. Pivot & Reshaping

```
# Pivot table (transform rows to columns)
pivot = df.pivot_table(
    values='salary',
    index='department',
    columns='year',
    aggfunc='mean'
)

# Melt (unpivot - columns to rows)
melted = df.melt(
    id_vars=['name'],
    value_vars=['q1', 'q2', 'q3']
)

# Stack/Unstack
stacked = df.set_index(['name', 'year'])['salary'].unstack()
```

11. String Operations

```
# On Series with strings
s = pd.Series(['Alice', 'Bob', 'Charlie'])

s.str.upper()          # Uppercase
s.str.lower()          # Lowercase
s.str.len()            # String length
s.str.contains('li')   # Boolean search
s.str.replace('a', 'x') # Replace
s.str.split(' ')       # Split into list
```

12. Basic Statistics

```
df.mean()              # Mean per column
df.median()            # Median
df.std()               # Standard deviation
df.var()               # Variance
df.sum()               # Sum
df.min() / df.max()    # Min/Max
df.corr()              # Correlation matrix
df.cov()               # Covariance matrix
```

13. Data Type Conversion

```
# Convert data type
df['age'] = df['age'].astype(int)
df['salary'] = df['salary'].astype(float)

# To datetime
df['date'] = pd.to_datetime(df['date'])

# Extract from datetime
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day_name'] = df['date'].dt.day_name()

# Categorical (reduces memory)
df['category'] = df['category'].astype('category')
```

14. Duplicates & Unique Values

```
# Check for duplicates
df.duplicated()           # Boolean Series
df.duplicated(subset=['name']) # By specific column

# Remove duplicates
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['name'])

# Unique values
df['name'].unique()       # Array of unique values
df['name'].nunique()      # Count of unique values
df['name'].value_counts() # Frequency table
```

15. Saving & Exporting

```
# Save to CSV
df.to_csv('output.csv', index=False)

# Save to Excel
df.to_excel('output.xlsx', index=False)

# Save to JSON
df.to_json('output.json')

# Save to SQL
df.to_sql('table_name', connection, if_exists='replace')

# Save to Parquet (fast, compressed)
df.to_parquet('output.parquet')
```

Quick Reference Cheat Sheet

Task	Code
Load CSV	<code>pd.read_csv('file.csv')</code>
First/Last rows	<code>df.head()</code> / <code>df.tail()</code>
Get column	<code>df['name']</code> or <code>df.loc[:, 'name']</code>
Filter rows	<code>df[df['age'] > 25]</code>
Group & aggregate	<code>df.groupby('dept')['salary'].mean()</code>
Merge	<code>pd.merge(df1, df2, on='id')</code>
Fill NaN	<code>df.fillna(0)</code>
Drop duplicates	<code>df.drop_duplicates()</code>
Pivot	<code>df.pivot_table(values='x', index='y', columns='z')</code>
Save to CSV	<code>df.to_csv('out.csv')</code>

Key Takeaways

1. **Series** = 1D data; **DataFrame** = 2D table
2. Use `.loc[]` for label-based, `.iloc[]` for position-based selection
3. Boolean indexing: `df[df['column'] > value]`
4. `.groupby()` + `.agg()` handles 80% of data analysis tasks
5. Always check `.info()` and `.describe()` first
6. Missing data? Use `.fillna()` or `.dropna()`
7. `.merge()` and `.concat()` combine datasets
8. Save with `.to_csv()`, `.to_excel()`, or `.to_parquet()`

Happy data wrangling!